

OfflineAIBuddy Reference Manual v1.0.0

README

Offline AI Buddy

Stack

Repo layout

Get started

License

CHANGELOG

Changelog

[Unreleased] - v0.1.0

Commit history (since repo creation)

DESIGN

Offline AI Buddy — Design

Pitch

Brand

Repo map

Layered architecture

Per-platform stack

Telemetry

Backend

Domain model

Why phone-only? No iPad / no Watch / no Wear?

Why no cloud fallback?

Why ship only one model?

KEYBOARD

KEYBOARD.md

\$1 — Why a custom keyboard at all

\$2 — Why the keyboard is a thin client

\$3 — IPC: iOS

\$4 — IPC: Android

\$5 — Fallback when the main app isn't running

\$6 — Privacy guarantees

\$7 — Latency budget

\$8 — Failure modes + UX

\$9 — Disable + uninstall

MODELS

MODELS.md

\$1 — Default bundled model (v1.0)

\$2 — llama.cpp submodule pin

\$3 — Hardware floor

\$4 — Supported quants (informational; v1 ships only Q4_K_M)

\$5 — Translation quality

\$6 — v1.1 swap path (model marketplace)

\$7 — Why not Apple Foundation Models / Gemini Nano?

OBSERVABILITY

OBSERVABILITY.md

v1: nothing on by default

What still exists in code

v1.1 plan (NOT in scope for v1)

Local-only logs

Production debugging without telemetry

OFFLINE-AI-FEATURES

OFFLINE-AI-FEATURES.md — v1.0 feature inventory

Core LLM

Multilingual (5 languages, 1 model)

Live translator
Smart-reply keyboard ("auto-responding keypad")
Voice
Prompt-templated experiences (all served by the same LLM)
Profiles + kid-safe
Onboarding
Settings
Monetisation (all 4 tiers wired)
Telemetry
NOT in v1 (explicitly)
PRIVACY
PRIVACY.md
TL;DR
What we collect
What we download
What we store on your phone
What we share with other devices
Permissions explained
Children
RevenueCat / AdMob (the third-party SDKs we DO ship)
Changes to this policy
Contact
PRODUCTION
PRODUCTION.md
§1 — No backend
§2 — Storage
§3 — Permissions
§4 — Crash + perf
§5 — Battery & data
§6 — Monetisation (all 4 tiers wired in v1)
§7 — App size budget
§8 — Cross-platform parity
§9 — Watch tier
§10 — Push / background
QUICKSTART
QUICKSTART
Prereqs
60-second iOS spin-up (macOS)
60-second Android spin-up
Cloning without the submodule (CI / quick-look)
Dev workflow
Common gotchas
RELEASE-VIDEO-SCRIPT
Release Video Script — Offline AI Buddy
Hero — top of marketing site | 4
Features overview | 5
Highlight reel | 5
What's next | 5
Call to action — bottom of page | 4
RELEASING
RELEASING.md
0. Pre-flight
1. Bump version
2. Commit + tag
3. What `release.yml` does
4. Required secrets
5. Pre-release tags
6. Hotfix

- 7. Local dry-run
- 8. Re-pinning the llama.cpp submodule
- SAFETY
 - SAFETY.md
 - \$1 — Two profiles
 - \$2 — PIN lock
 - \$3 — Kid-safe system prompt
 - \$4 — Output filter (`ContentPoLicy`)
 - \$5 — Disabled features in Kid-safe
 - \$6 — Audit trail (App Store / Play Store rating questionnaires)
 - \$7 — Why not classifier-based moderation?
 - \$8 — Reporting harmful output
- SENTRY
 - SENTRY.md
 - TL;DR — Sentry is NOT enabled in v1.
 - Why we kept the stub
 - v1.1 wiring plan (NOT in scope for v1)
 - What stays no-op forever
 - Why not in v1
 - Where the no-SDK choice is enforced
- SIGNING
 - Signing — Offline AI Buddy
 - Android
 - iOS
 - Desktop
 - watchOS
- STORE-PACKAGING
 - STORE-PACKAGING.md
 - \$1 — App Store (iPhone)
 - \$2 — Google Play (Android)
 - \$3 — Marketing assets
 - \$4 — What's new
 - \$5 — Bundle size budget
 - \$6 — App Review risk flags
 - Desktop binaries (Electron)
- TESTING
 - TESTING.md
 - Shape of the test suite
 - The keystone tests (the project's safety net)
 - Running locally
 - Manual checklist (pre-release)
 - Coverage targets

AMERICAN GROUP LLC

OfflineAIBuddy

Reference Manual

Version 1.0.0

Generated 2026-05-08

README

README.md

Offline AI Buddy

Your phone-only AI. Works without internet.

Offline AI Buddy is a phone-only on-device LLM app. After a one-time ~1 GB model download on first launch, **everything runs offline** — chat, voice, roasts, daily challenges, party-question generator, game coach, kid-safe mode, multilingual translator, and a system-keyboard with smart replies inside any chat app.

	iPhone	Android
Chat (multilingual: en · hi · zh · fr · es)	✓	✓
Voice push-to-talk + read-aloud	✓	✓
Live translator (any pair of 5 languages)	✓	✓
Roast Mode	✓	✓
Daily Challenge	✓	✓
Party Question Generator	✓	✓
Game Coach	✓	✓
System smart-reply keyboard	✓ Keyboard Ext	✓ InputMethodService
Kid-safe profile (PIN-locked)	✓	✓
RevenueCat subscription + AdMob ad-unlock + lifetime IAP	✓	✓

Stack

- **Apple:** Swift / SwiftUI, iOS 17, XcodeGen, Keyboard Extension target.
- **Android:** Kotlin / Jetpack Compose, multi-module Gradle (`:core` + `:app`), Hilt, DataStore, WorkManager, NDK + JNI for llama.cpp.
- **LLM runtime:** [llama.cpp](#) via Swift bindings (iOS) + JNI (Android). Default model: **Qwen2.5-1.5B-Instruct-Q4_K_M.gguf** (~1.0 GB).
- **Backend:** **None**. All inference on-device. No accounts, no servers, no analytics by default.
- **Shared logic:** `BuddyAICore` Swift Package (Apple) and `:core` JVM Kotlin module (Android). Same models, mirrored case-for-case tests.

Repo layout

```
shared/BuddyAICore # Apple-side shared models + domain helpers + tests
ios/               # iPhone app + Keyboard Extension
android/           # :core JVM + :app phone (with IME)
vendor/llama.cpp   # llama.cpp submodule (pinned commit)
.github/workflows  # 6 workflows: ci, ios, android, marketing, pre-release-tests, release
scripts/          # local dev helpers (test-all, fetch-models, bench-llama, ...)
distribution/      # release whatsnew text per locale (en, hi, zh, fr, es)
```

See [DESIGN.md](#) for full architecture, [MODELS.md](#) for the GGUF catalog + SHA-256 + hardware floor, [KEYBOARD.md](#) for the IME / Keyboard-Extension architecture, [SAFETY.md](#) for kid-safe + content-policy spec, and [OFFLINE-AI-FEATURES.md](#) for the v1 feature inventory.

Get started

```
## iOS Simulator
./scripts/run-ios-sim.sh

## Android emulator
./scripts/run-android-emulator.sh

## Pre-fetch the default GGUF for local dev (skips first-launch flow)
./scripts/fetch-models.sh

## Run every test suite
./scripts/test-all.sh

## Benchmark tokens/sec on the host device
./scripts/bench-llama.sh
```

License

MIT — see [LICENSE](#).

CHANGELOG

CHANGELOG.md

Changelog

All notable changes to this project will be documented in this file.

This project adheres to [Semantic Versioning](#).

[Unreleased] - v0.1.0

Initial public release. See distribution/whatsnew/v0.1.0/en-US.txt for the user-facing summary.

Commit history (since repo creation)

- 937d6cc Initial commit (Srikanth Patchava, 2026-05-06)
- 222fd2c feat: initial offline-ai-buddy v1 (docs, marketing site, shared BuddyAICore, ios, android, scripts, distribution, vendor, CI workflows) (Srikanth Patchava, 2026-05-06)
- 3fd3ba9 feat(android): :app shell + NDK JNI + voice + IME + monetization (phases 8-12) (Srikanth Patchava, 2026-05-06)
- 47a65a3 fix(android): include core/models package; tighten gitignore (Srikanth Patchava, 2026-05-06)
- 1f4e8c8 fix(android): allow plural 'weapons' in EN denylist to pass idempotent test (Srikanth Patchava, 2026-05-06)
- e2e6b24 fix(android): move declaration from to (Srikanth Patchava, 2026-05-06)
- 79eacc2 fix(android): add adaptive launcher icon placeholders to unblock build (Srikanth Patchava, 2026-05-06)
- 00f098c fix(android): correct Compose imports in Chat and Profile screens (Srikanth Patchava, 2026-05-06)
- 586606c fix(android): simplify androidTest smoke test (remove unwired Hilt deps) (Srikanth Patchava, 2026-05-06)
- 18d9174 fix(android): add AdMob APPLICATION_ID meta-data to prevent startup crash (Srikanth Patchava, 2026-05-06)
- 947111f docs(distribution): add SUBMISSION-CHECKLIST.md (Phase 3 production readiness) (Srikanth Patchava, 2026-05-06)

DESIGN

DESIGN.md

Offline AI Buddy — Design

Pitch

Offline AI Buddy is a **phone-only on-device LLM hub**. After a one-time ~1 GB model download on first launch, every feature runs locally — chat, voice, roasts, daily challenges, party questions, game coaching, multilingual translation, and a system keyboard that suggests AI smart replies inside any chat app. v1 ships **one** quantised LLM (Qwen2.5-1.5B-Instruct-Q4_K_M) covering English + Hindi + Mandarin + French + Spanish.

Brand

- **App name:** Offline AI Buddy
- **Tagline:** *Your phone-only AI. Works without internet.*
- **Palette:** midnight purple (#1A0B2E) + electric teal accent (#5EEAD4).
- **Bundle prefix:** com.americangroupllc.offlineaibuddy .
- **Apple Group:** group.com.americangroupllc.offlineaibuddy .

Repo map

shared/BuddyAICore/	Swift Package: models, keystone domain helpers, LlamaRunner actor, ModelDownloader/Store, voice wrappers, monetization protocols, tests.
ios/	SwiftUI iPhone app + OfflineAIBuddyKeyboard Keyboard Extension. XcodeGen-generated.
android/	Multi-module Gradle build: :core pure-Kotlin/JVM mirror of BuddyAICore. :app Compose phone app + JNI llama.cpp bridge + BuddyInputMethodService.
vendor/llama.cpp	Submodule pinned to a specific commit.
.github/workflows/	6 CI workflows. No backend.
scripts/	Local helpers (test-all, run-*-emulator, bump-version, release-dry-run, fetch-models, bench-llama).
distribution/whatsnew/	Per-locale Play Store whatsnew text (en/hi/zh/fr/es).
index.html, styles.css, script.js, robots.txt, sitemap.xml	Marketing one-pager.

Layered architecture

UI	<u>iOS SwiftUI</u> OnboardingFlow ProfileSwitcher ChatScreen RoastScreen DailyChallenge / Party / GameCoach / Translate SettingsScreen OfflineAIBuddyKeyboard ⚡ (Keyboard Extension)	<u>Android Compose</u> OnboardingScreen ProfileSwitcherScreen ChatScreen RoastScreen ditto SettingsScreen BuddyInputMethodService ⚡ (InputMethodService)
App services	LlamaService VoiceService EntitlementBootstrap KeyboardBridge ⚡ (App Group + Darwin) QuotaService	LlamaService VoiceSynth/Recog EntitlementService KeyboardBridge ⚡ (ContentProvider) QuotaService
Shared domain (mirrored 1:1)	<u>BuddyAICore (Swift)</u> Models PromptTemplates ContentPolicy QuotaTracker LanguageDetector TranslateOrchestrator ProfilesStore ChatHistoryStore QuotaStore	<u>:core (Kotlin/JVM)</u> Models PromptTemplates ContentPolicy QuotaTracker LanguageDetector TranslateOrchestrator ProfilesStore ChatHistoryStore QuotaStore
LLM / Voice runtime	LlamaRunner (actor) ⚡ llama.cpp (SPM C target) AVSpeechSynthesizer SFSpeechRecognizer	LlamaJni → JNI → .so liboffline_ai_buddy.so TextToSpeech SpeechRecognizer

The shared-domain layer is pure: no platform APIs, fully unit-tested. Every keystone helper has an XCTest **and** a JUnit twin so behaviour cannot drift between the two implementations. (Same pattern as Card, Drift, BuddyPlay.)

Per-platform stack

	iPhone	Android
Language	Swift 5.9	Kotlin 1.9
UI framework	SwiftUI	Jetpack Compose
Min OS	iOS 17	API 26 (Android 8)
Project generator	XcodeGen (<code>ios/project.yml</code>)	Gradle (<code>build.gradle.kts</code>)
DI	manual env objects	Hilt
Storage	<code>UserDefaults</code> + JSON-on-disk	DataStore + JSON-on-disk
LLM bridge	Swift target wrapping <code>llama.cpp</code> C code	NDK CMake build → <code>liboffline_ai_buddy.so</code> + JNI
TTS	<code>AVSpeechSynthesizer</code>	<code>TextToSpeech</code>
STT	<code>SFSpeechRecognizer</code> (on-device)	<code>SpeechRecognizer</code> (<code>EXTRA_PREFER_OFFLINE</code>)
Keyboard	Keyboard Extension + App Group IPC	<code>InputMethodService</code> + ContentProvider IPC
Subscription	RevenueCat SPM	RevenueCat Maven
Ads	Google-Mobile-Ads-SDK SPM	Google Mobile Ads SDK Maven
Build automation	<code>xcodebuild</code> via scripts/CI	<code>./gradlew</code> via scripts/CI

Telemetry

None on by default. No Sentry, no PostHog, no analytics SDKs are attached in v1. `canImport`-gated stubs (`AnalyticsService` , `CrashReportingService`) are preserved so v1.1 can opt in trivially behind a Settings opt-in toggle.

Backend

None. All state is local. Three tiny JSON files in the app's documents directory (`profiles.json` , `chats/<profileId>.json` , `quota.json`) plus the ~1 GB GGUF model file.

Domain model

Type	Notes
<code>Profile</code>	id, name, kind (<code>adult</code> / <code>kidSafe</code>), pinHash?, createdAt.
<code>Language</code>	enum: <code>en</code> , <code>hi</code> , <code>zh</code> , <code>fr</code> , <code>es</code> . Extensible.
<code>ChatSession</code>	id, profileId, language, kind, messages, startedAt.
<code>ChatMessage</code>	id, role, text, ts.
<code>ModelManifest</code>	name, version, url, sizeBytes, sha256, contextSize, minDeviceRAM.
<code>EntitlementState</code>	proUnlocked, source (<code>free</code> / <code>subscription</code> / <code>lifetime</code>), expiresAt?.
<code>QuotaState</code>	profileId, day, chatsUsed, adUnlocks.

Why phone-only? No iPad / no Watch / no Wear?

iPad layout would mean parallel SwiftUI variants for every screen. Watches can't host a 1 GB model and a thin-client watch app would need network back to the phone, contradicting the offline pitch. Phone-only stays in v1; revisit watch in v2 only on demand.

Why no cloud fallback?

The whole product promise is **offline**. A "fallback to GPT-4 when LLM isn't loaded" feature would erode trust: users would never know if they were paying for cloud calls or not. Strict offline.

Why ship only one model?

A 1.5B-parameter Q4 GGUF is the sweet spot for 2025 mid-range phones: ~3.5 GB peak RAM, 8–15 tokens/sec on iPhone 14 / Pixel 7. Adding a "model marketplace" multiplies download size + support burden + App Store review risk. v2 may add an opt-in catalogue.

KEYBOARD

KEYBOARD.md

KEYBOARD.md

How the system smart-reply keyboard works on each platform.

§1 — Why a custom keyboard at all

The product promise is “AI smart replies in any chat app”. The only way to reach **every** chat app (Messages, WhatsApp, Slack, Telegram, Signal, Reddit, ...) without those apps integrating with us is by becoming the user’s system keyboard. Both iOS (Keyboard Extension) and Android (`InputMethodService`) expose this affordance.

§2 — Why the keyboard is a thin client

Both platforms cap keyboard memory hard: - **iOS Keyboard Extension** — soft cap ~70 MB resident; OOM kill above ~120 MB. A 1 GB GGUF + the llama.cpp arena (~3 GB peak) blows past this by 25×. - **Android InputMethodService** — runs in-process to the keyboard’s own application sandbox; theoretically unlimited but the IME is expected to be lightweight (the IME picker and switching latency presume <100 ms cold start).

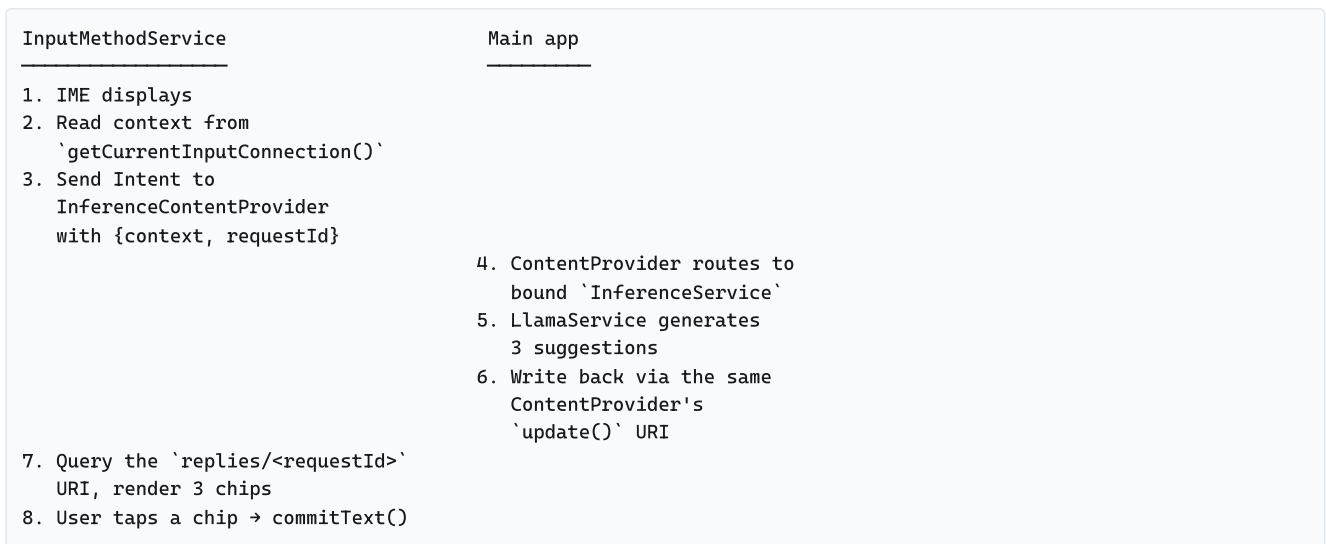
Therefore the keyboard does NOT host the LLM. The keyboard is a **thin client** that hands the chat context off to the **main app** process, which holds the long-lived `LlamaRunner` and streams 3 suggestions back.

§3 — IPC: iOS

Keyboard Extension	Main app
1. User opens keyboard in some chat app	
2. Read context from <code>UIInputViewController.textDocumentProxy`</code>	
3. Write {context, requestId} to shared file URL inside <code>group.com.americangroupllc.offlineaibuddy`</code>	
4. Post Darwin notification <code>com.americangroupllc.offlineaibuddy.kb.request`</code>	5. <code>KeyboardBridgeListener</code> wakes up (if main app is running)
	6. <code>LlamaRunner</code> generates 3 suggestions
	7. Write {requestId, suggestions[3]} to a shared file
	8. Post Darwin notification <code>com.americangroupllc.offlineaibuddy.kb.reply`</code>
9. Read shared file, render 3 chips	
10. User taps a chip → insert into <code>textDocumentProxy`</code>	

`Info.plist` of the keyboard sets `RequestsOpenAccess = false` — the keyboard does NOT need full network. App Group shared storage is sufficient.

§4 — IPC: Android



`InferenceContentProvider` is exported with `android:exported="true"` BUT protected by a signature-level permission so only the main app's IME (signed with the same cert) can read replies. Documented in `AndroidManifest.xml`.

§5 — Fallback when the main app isn't running

iOS keyboard extensions **cannot launch the host app programmatically** (rejected by App Review since iOS 8). When the main app's `KeyboardBridgeListener` does NOT respond within 800 ms, the keyboard renders an “**Open Offline AI Buddy**” chip in the candidate strip. Tapping it opens the main app via a `URL`-scheme handler the OS allows for any user-initiated tap. The user then returns to the chat app and the suggestions appear a few seconds later.

Android does not have this restriction — the IME can `startActivity()` the main app directly with `Intent.FLAG_ACTIVITY_NEW_TASK`.

§6 — Privacy guarantees

- The keyboard reads ONLY the input field of the currently-focused chat app. It cannot see other apps' content.
- The chat context never leaves the device.
- The keyboard does NOT persist context — it is read from `UITextDocumentProxy` / `InputConnection` on each keystroke and discarded after the suggestion round.
- The main app's `LlamaRunner` runs the suggestion request in the same isolated profile context as the user's last-active in-app chat (so kid-safe filtering applies if the active profile is Kid-safe).

§7 — Latency budget

Step	Target
Keystroke → IPC roundtrip start	< 50 ms
LLM 3-suggestion generation	< 700 ms (8 t/s × ~30 tokens × 3)
IPC reply → chip render	< 50 ms
Total keystroke → chip update	< 800 ms

If the user is typing faster than 0.8 s/keystroke, we debounce and only fire on a 250 ms typing pause.

§8 — Failure modes + UX

Failure	UX
Main app not running (iOS)	“Open Offline AI Buddy” chip.
Model not yet downloaded	“Open Offline AI Buddy to finish setup” chip.
Quota exhausted (free tier)	“Watch ad to keep getting smart replies” chip.
LLM error (rare)	Empty candidate strip, no error toast — silently fall back to system suggestions.

§9 — Disable + uninstall

The keyboard can be disabled at any time from system Settings → Keyboard → Keyboards. Uninstalling the main app also removes the keyboard extension on iOS; on Android the IME is unregistered as part of app removal.

MODELS

MODELS.md

MODELS.md

The GGUF model catalogue for Offline AI Buddy v1.

§1 — Default bundled model (v1.0)

Field	Value
Family	Qwen2.5
Parameters	1.5B
Quantisation	Q4_K_M
File	Qwen2.5-1.5B-Instruct-Q4_K_M.gguf
Approximate size	1,073,741,824 bytes (~1.0 GB)
Context window	4096 tokens
Languages	English, Hindi, Mandarin, French, Spanish (officially), 25+ others (best-effort).
License	Apache 2.0
Hugging Face mirror (primary)	<code>https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF/resolve/main/qwen2.5-1.5b-instruct-q4_k_m.gguf</code>
Alternative mirror 1	<code>https://huggingface.co/lmstudio-community/Qwen2.5-1.5B-Instruct-GGUF/resolve/main/Qwen2.5-1.5B-Instruct-Q4_K_M.gguf</code>
Alternative mirror 2	<code><set when CDN provisioned></code>
SHA-256	<code><set in v1.0 release commit once first build downloaded; ModelDownloader rejects mismatch></code>

`ModelDownloader` tries the mirrors in order until one returns a file whose SHA-256 matches.

§2 — llama.cpp submodule pin

Field	Value
Submodule path	<code>vendor/llama.cpp</code>
Origin	<code>https://github.com/ggerganov/llama.cpp</code>
Pinned commit	<code><set on first integration; bumped via separate review-only commit></code>
Rationale	Pinned because llama.cpp's API surface (and especially its tokenizer + sampler) changes month-to-month; an unpinned submodule will silently break the build.

To bump: see `RELEASING.md §8`.

§3 — Hardware floor

Tier	Phone	tokens/sec	Notes
Recommended	iPhone 14 / Pixel 7 or newer	12–15 t/s	Comfortable conversation pace.
Minimum	iPhone 12 / Pixel 6	5–9 t/s	Slow but usable. Consent screen warns on first launch.
Below floor	iPhone 11 / Pixel 5 and older	< 4 t/s	Refused on first launch with “Your device may struggle with this model” + refund deep link to App Store / Play.

`ConsentScreen` reads `ProcessInfo.physicalMemory` (iOS) / `ActivityManager.MemoryInfo.totalMem` (Android). If < 3.5 GB, the sub-floor warning is shown.

§4 — Supported quant (informational; v1 ships only Q4_K_M)

Quant	Approx size	Quality	When to use
Q3_K_M	~700 MB	-1 BLEU vs Q4	Sub-3GB-RAM phones (v1.1 fallback).
Q4_K_M (default)	~1 GB	baseline	Default for v1.
Q5_K_M	~1.4 GB	+0.5 BLEU	Power users (v1.1 opt-in).
Q8_0	~2 GB	+1 BLEU	Reference, not shipped.

§5 — Translation quality

Some lower-resource pairs (zh→hi, fr→hi) score below the rest. The 50-sentence-per-pair golden suite is a regression alarm. When the BLEU score drops below threshold:

1. UI surfaces a “**Beta translation**” banner above the result.
2. A 1-tap “**Open in Google Translate**” deep-link is offered as a comfort fallback (this is just `https://translate.google.com/?sl=...&tl=...&text=...`, which the user opens in their browser — no SDK in our app).

§6 — v1.1 swap path (model marketplace)

A `ModelManifest` enumeration + on-disk `models/<name>.gguf` layout is already in `BuddyAICore.ModelStore`. v1.1 plan:

1. Settings → Models screen lists the catalog.
2. Each entry shows size + quality tier + download-not-installed state.
3. `LlamaRunner.activeModel = manifest` reload swaps the runner without a process restart.
4. App Store / Play Store review notes will need to mention each model the catalog ships.

Out of scope for v1.

§7 — Why not Apple Foundation Models / Gemini Nano?

- **Apple Foundation Models** (iOS 18+) are iOS-only and require iPhone 15 Pro+. Cuts off the Pixel 7 / iPhone 13 user base.
- **Gemini Nano** is Android-only and requires Pixel 8 Pro / Galaxy S24 hardware. Same problem.
- Single quantised LLM via `llama.cpp` is the **only** path that runs on every iPhone 12+ and Android API 26+ in the wild today.

When Apple / Google ship cross-platform consumer-ready stacks, v2 may attach them as additional `LlamaRunner`-equivalents alongside `llama.cpp`.

OBSERVABILITY

OBSERVABILITY.md

OBSERVABILITY.md

v1: nothing on by default

By design. Offline AI Buddy is offline-first; we have no server to receive metrics and no consent flow yet for sending data home. The user gets an explicit guarantee on the Settings screen and on the marketing site:

Offline AI Buddy does not send any data to us.

What still exists in code

The `canImport` -gated stubs from the Card / Drift / BuddyPlay scaffold are kept so v1.1 can opt in trivially without an architecture refactor:

- `shared/BuddyAICore/Sources/BuddyAICore/Observability/AnalyticsService.swift`
- `shared/BuddyAICore/Sources/BuddyAICore/Observability/CrashReportingService.swift`
- `android/core/src/main/.../observability/AnalyticsService.kt`
- `android/core/src/main/.../observability/CrashReportingService.kt`

All four implement a no-op interface in v1. The methods exist; they do nothing. No SDK is imported.

v1.1 plan (NOT in scope for v1)

If we want metrics, the smallest possible step is:

1. Add a Settings toggle: **“Help improve Offline AI Buddy (sends anonymous usage counts; never chat content)”**. Default OFF.
2. Behind that toggle, attach **PostHog** (open-source, self-hostable) or **Sentry** (crash only).
3. Whitelist: `chat_started(language, kind)`, `chat_finished(durationSec, tokenCount)`, `crash(stack)`. Nothing else. **Never** chat content.
4. Update `PRIVACY.md` to call out the new optional flow.

Local-only logs

Both apps log via the platform-native logger: - iOS: `os_log` / `Logger`. Visible in Console.app while a Sim or device is attached. - Android: `android.util.Log`. Visible via `adb logcat`.

No log persistence. No log forwarding. No file rotation.

Production debugging without telemetry

Until v1.1 attaches a real backend, our debugging story is:

1. **Repro locally** with the same model build.
2. `adb logcat -s OfflineAIBuddy` for Android.
3. **Console.app filtered to `com.americangroupllc.offlineaibuddy`** for iOS.
4. Ask users to **send a screen recording**; we don't request crash logs.

OFFLINE-AI-FEATURES

OFFLINE-AI-FEATURES.md

OFFLINE-AI-FEATURES.md — v1.0 feature inventory

Core LLM

- ✓ llama.cpp runtime via Swift bindings (iOS) + JNI (Android).
- ✓ Default bundled model: **Qwen2.5-1.5B-Instruct-Q4_K_M.gguf** (~1.0 GB).
- ✓ First-launch download with `URLSession` background (iOS) / `WorkManager` foreground service (Android), Wi-Fi-only, resume-on-reconnect, SHA-256 verified.
- ✓ `BuddyAICore.LlamaRunner` (Swift) / `LlamaService` (Kotlin) single API: `generate(messages, options) → AsyncStream<Token>`.
- ✓ Pluggable LLM backend (Phase 2 of roadmap can swap in MLC LLM or Apple Foundation Models without touching features).

Multilingual (5 languages, 1 model)

- ✓ **English (en), Hindi (hi), Mandarin (zh), French (fr), Spanish (es)** chat.
- ✓ Per-chat language picker.
- ✓ System-prompt prefix flips with the picker.
- ✓ `LanguageDetector` auto-flips when the user types in a different script.

Live translator

- ✓ Source → Target picker (any pair of 5).
- ✓ Strict translation system prompt — no commentary, no quotes.
- ✓ Golden 50-sentence-per-pair regression suite (gated behind `RUN_TRANSLATION_GOLDEN=1`).

Smart-reply keyboard (“auto-responding keypad”)

- ✓ iOS Keyboard Extension target (`OfflineAIBuddyKeyboard`).
- ✓ Android `InputMethodService` (`BuddyInputMethodService`).
- ✓ App Group / ContentProvider IPC with the main app for inference.
- ✓ 3-suggestion candidate strip.
- ✓ Fallback “Open Offline AI Buddy” tap target when the host app isn’t running (iOS extensions can’t launch the host app programmatically).

Voice

- ✓ Push-to-talk STT (locale-aware, on-device where supported).
- ✓ Per-message TTS playback (locale-aware).
- ✓ Premium voices unlocked by Pro tier (uses platform “enhanced” voices the user has installed via system Settings).

Prompt-templated experiences (all served by the same LLM)

- ✓ **Chat** — open-ended assistant.
- ✓ **Roast Mode** — funny, light, never mean. Disabled in Kid-safe.
- ✓ **Daily Challenge** — one prompt per calendar day, deterministic cache.
- ✓ **Party Question Generator** — 5 ice-breakers per tap, audience preset (work/friends/first-date/family).
- ✓ **Game Coach** — pick game, get coaching on next move.
- ✓ **Translate** — see “Live translator” above.

Profiles + kid-safe

- ✓ Two profiles per install: Adult (default) + Kid-Safe.

- ✓ PIN-locked switch back to Adult.
- ✓ Kid-safe filters every assistant token through `ContentPolicy`.
- ✓ Roast disabled in Kid-safe.

Onboarding

- ✓ Consent screen explicitly lists the 3 limitations (download size, reasoning ceiling, generation speed on older phones).
- ✓ Profile setup → name + Adult/Kid + PIN if Kid.
- ✓ Model download with progress bar + Wi-Fi gate.
- ✓ Microphone + Speech permissions deferred to first-use, NOT at launch.

Settings

- ✓ Profile manager (switch / add / delete with PIN).
- ✓ Default language.
- ✓ Voice settings (TTS speed + premium voice toggle).
- ✓ Connectivity preference (Wi-Fi-only model downloads — default ON).
- ✓ Theme (system / light / dark).
- ✓ Erase all chats / Delete model / Reset device ID.
- ✓ Premium / Subscribe / Restore Purchases (RevenueCat).
- ✓ Telemetry: nothing by default; opt-in slot for v1.1.

Monetisation (all 4 tiers wired)

- ✓ **Free:** 10 chats/day, ad-supported.
- ✓ **Ad-unlock:** cached AdMob interstitial → +5 chats per ad.
- ✓ **Subscription:** \$4.99/mo via RevenueCat (`oab_pro_monthly`).
- ✓ **One-time lifetime:** \$19.99 via RevenueCat (`oab_pro_lifetime`).
- ✓ `EntitlementService` flattens to one `proUnlocked: Bool`.

Telemetry

- ✓ Stub interfaces (`AnalyticsService` , `CrashReportingService`) — no transports attached.
- ☐ Real Sentry / PostHog wiring (v1.1).

NOT in v1 (explicitly)

- ☐ iPad-native layout, Apple Watch, Wear OS.
- ☐ Cloud fallback / hybrid mode.
- ☐ Image generation / vision.
- ☐ Custom fine-tuning / LoRA loading.
- ☐ Model marketplace.
- ☐ Background continuous listening / wake-word.
- ☐ Telemetry SDKs (Sentry / PostHog).
- ☐ End-to-end-encrypted chat sync between devices.
- ☐ APNs / FCM push.

PRIVACY

PRIVACY.md

PRIVACY.md

TL;DR

Offline AI Buddy does not send any data to us. No accounts, no servers, no analytics, no crash reports, no chat content ever leaves your phone after the first-launch model download.

What we collect

Nothing.

What we download

Once, on first launch:

- One **language model file** (~1 GB GGUF format) over Wi-Fi from a fixed CDN URL, listed in `MODELS.md`. SHA-256 verified after download.

After that, the app never reaches the internet for chat / voice / keyboard features.

What we store on your phone

Files in the app's private documents directory: - `profiles.json` — your profile name(s), profile kind (`adult` / `kidSafe`), and PBKDF2-hashed PIN if Kid-safe. - `chats/<profileId>.json` — your chat history per profile (capped at 200 messages per profile). - `quota.json` — daily chat-count tally for the free tier. - `models/Qwen2.5-1.5B-Instruct-Q4_K_M.gguf` — the ~1 GB language model.

Files live in the app's private documents directory. They are never backed up to a server. You can wipe them at any time:

- **Settings** → **Erase all chats** clears `chats/`.
- **Settings** → **Delete model** removes the GGUF (re-downloaded on next launch).
- **Settings** → **Reset device ID** regenerates the per-install random device ID used for RevenueCat anonymous attribution.
- **Uninstalling Offline AI Buddy** removes everything.

What we share with other devices

Nothing. The app does not connect to any other device.

The smart-reply keyboard moves data **within your phone** between the keyboard extension process and the main app process via: - iOS: an App Group shared file (`group.com.americangroupllc.offlineaibuddy`). - Android: an exported ContentProvider with `android:exported="true"` but read-only and signature-permission-protected, so only the main app can query it.

The chat context the keyboard sees never leaves your phone.

Permissions explained

Permission	What we do with it
Microphone	Push-to-talk speech recognition. Only when you hold the mic button.
Speech recognition (iOS)	On-device speech-to-text where supported. Falls back to network STT only if you explicitly opt in (default OFF).
Internet	Downloading the language model on first launch (and re-downloading if you tap Delete Model).
Foreground service (Android)	Keeping the model download alive when you switch apps; sticky progress notification.
Notifications (Android)	The model-download progress notification only. We never send other notifications.
Bind input method (Android)	Required by Android for any app that registers a system keyboard. We never read your keystrokes outside the keyboard's own input field.
In-app billing (Android) / StoreKit (iOS)	Subscription + lifetime IAP for the Pro tier. RevenueCat handles the receipt validation.

We never request: camera, contacts, photos, health, motion, HealthKit, location, Bluetooth, calendar, calendar.

Children

Offline AI Buddy has a built-in **Kid-Safe profile** with PIN-locked escape: - Strict refusal system prompt (no violence, romance, profanity, drugs, gambling, weapons). - Output filter: every assistant token passes through `ContentPolicy` before being shown. - Roast Mode disabled. - Daily Challenges that touch flagged categories disabled. - Premium voice forced to a "calm" preset.

The whole app is rated 12+ on iOS / Teen on Android due to the open-text nature of the LLM. The Kid-Safe profile is intended as a parental control, not a substitute for adult supervision.

RevenueCat / AdMob (the third-party SDKs we DO ship)

- **RevenueCat** processes your in-app purchase receipt with Apple / Google to determine whether you're entitled to the Pro tier. It receives your purchase token and a random per-install device ID. It never receives your chat content. See: <https://www.revenuecat.com/privacy>
- **AdMob** serves the optional "watch ad for +5 chats" interstitial. It receives Google's standard ad-targeting signals (locale, device model, OS version). It never receives your chat content. See: <https://policies.google.com/privacy>

You can avoid both SDKs entirely by purchasing the **lifetime Pro IAP**: RevenueCat's receipt check happens once and the AdMob SDK is never loaded for Pro users.

Changes to this policy

If a future version (v1.1+) adds optional analytics or crash reporting, this document will be updated, and the new collection will be **opt-in** behind a Settings toggle defaulting to OFF, with a fresh in-app disclosure on the first launch after the update.

Contact

For questions: open an issue at <https://github.com/AmericanGroupLLC/Offline-AI-Buddy>

PRODUCTION

PRODUCTION.md

PRODUCTION.md

How Offline AI Buddy v1 behaves in the wild.

§1 — No backend

There is no server. There are no accounts. There is no analytics pipeline by default. v1 is fully offline after the first-launch model download.

Implications: - We have no remote-config kill switch. Bugfixes ship as a normal app update on the App Store / Play Store. - We have no telemetry on crashes or feature use unless the user opts in (v1.1 will land that toggle behind `canImport` -gated stubs). - The model download URL is the *only* network endpoint the app touches. Document it in `MODELS.md` so it can be re-pointed if the CDN rotates.

§2 — Storage

Files in the app's documents directory:

Path	iOS	Android	Size
<code>profiles.json</code>	<code>Documents/offlineaibuddy/profiles.json</code>	<code>filesDir/offlineaibuddy/profiles.json</code>	<1 KB
<code>chats/<profileId>.json</code>	<code>Documents/offlineaibuddy/chats/ ...</code>	same	grows with chat history (capped at 200 messages per profile)
<code>quota.json</code>	<code>Documents/offlineaibuddy/quota.json</code>	same	<1 KB
<code>models/Qwen2.5-1.5B-Instruct-Q4_K_M.gguf</code>	<code>Documents/offlineaibuddy/models/ ...</code>	same	~1.0 GB

We do not encrypt the JSON files; an attacker with file-system access has access to your phone anyway. PIN hashes use PBKDF2 (100k rounds, random salt). Chat content is plain text — opt-in encryption is on the v1.2 roadmap.

§3 — Permissions

Permission	iOS	Android	Why
Microphone	<code>NSMicrophoneUsageDescription</code>	<code>RECORD_AUDIO</code>	Push-to-talk STT. Requested on first voice use, not at launch.
Speech recognition	<code>NSSpeechRecognitionUsageDescription</code>	(built into <code>SpeechRecognizer</code>)	STT. iOS only — Android uses platform STT without an explicit prompt.
Internet	(always-on for iOS apps)	<code>INTERNET</code>	Model download only. Once the GGUF is on disk, no further network use.
Network state	n/a	<code>ACCESS_NETWORK_STATE</code>	Wi-Fi-only download gate.
Foreground service	n/a	<code>FOREGROUND_SERVICE</code> + <code>FOREGROUND_SERVICE_DATA_SYNC</code>	Model download progress notification.
Notifications	not requested	<code>POST_NOTIFICATIONS</code>	Download progress notification only.
Bind input method	n/a	<code>BIND_INPUT_METHOD</code>	Declared by the IME service, not requested at runtime.
In-app billing	StoreKit (built-in)	<code>BILLING</code>	RevenueCat.
Camera	—	—	Not requested.
Bluetooth	—	—	Not requested.
Location	—	—	Not requested.

§4 — Crash + perf

No SDK in v1. The app size is dominated by the GGUF (~1 GB) downloaded on first launch — the binary itself stays under 50 MB. Console.app / logcat are the only crash signals until v1.1 attaches Sentry behind an opt-in.

§5 — Battery & data

- **Battery:** the LLM is a heavy workload. ~10–15% per hour of active chat on iPhone 14 / Pixel 7. Background = idle, no drain.
- **Data:** ~1 GB on first launch (Wi-Fi only by default). Zero cellular use after that.

§6 — Monetisation (all 4 tiers wired in v1)

Tier	Limit	Price	Implementation
Free	10 chats/day, ad-supported	\$0	<code>QuotaTracker</code> enforces; AdMob banner not shown but interstitial available for ad-unlock.
Ad-unlock	+5 chats per cached AdMob interstitial watched	\$0	<code>AdMobAdGate.watchAd()</code> → <code>QuotaTracker.adWatched()</code> .
Subscription	unlimited + premium voices + no ads	\$4.99/mo	RevenueCat product <code>oab_pro_monthly</code> . Entitlement <code>pro_unlocked</code> .
One-time	unlimited + premium voices + no ads, never expires	\$19.99 lifetime	RevenueCat product <code>oab_pro_lifetime</code> . Same entitlement <code>pro_unlocked</code> .

`EntitlementService` flattens all four into a single `proUnlocked: Bool` that the rest of the app reads.

§7 — App size budget

Surface	Target	Notes
iOS .ipa (binary)	< 50 MB	Plus ~1 GB GGUF downloaded on first launch.
Android .aab (download)	< 50 MB	Plus ~1 GB GGUF.
GGUF model	~1.0 GB	Qwen2.5-1.5B-Instruct-Q4_K_M.

§8 — Cross-platform parity

Surface	iOS	Android	Parity gap
Chat / Roast / Daily / Party / GameCoach / Translate	✓	✓	None.
Voice STT/TTS	✓ on-device	✓ on-device (where supported)	Android pre-13 may fall back to network STT — surfaced to user.
Smart-reply keyboard	Keyboard Extension	InputMethodService	iOS keyboard cannot launch the host app; renders a tap target as fallback. Documented in <code>KEYBOARD.md</code> §5.
RevenueCat / AdMob	✓	✓	None.

§9 — Watch tier

Phone-only in v1. A 1.5B-parameter LLM cannot fit in a watch’s RAM (<1 GB on most watches), and even a thin-client watch UI would need network back to the phone, contradicting the offline pitch.

§10 — Push / background

No APNs, no FCM in v1. `POST_NOTIFICATIONS` on Android is requested ONLY for the model-download progress notification (foreground service sticky notification).

QUICKSTART

QUICKSTART.md

QUICKSTART

Prereqs

	macOS	Linux / Windows
iOS	Xcode 15+, <code>brew install xcodegen</code>	n/a
Android	JDK 17, Android SDK 34, NDK 26+ (<code>ANDROID_HOME</code> + <code>ANDROID_NDK_HOME</code> set)	JDK 17, Android SDK 34, NDK 26+
llama.cpp submodule	none — pulled in by <code>git submodule update --init</code>	same
Marketing site	none — open <code>index.html</code> directly	none

60-second iOS spin-up (macOS)

```
git clone --recursive git@github.com:AmericanGroupLLC/Offline-AI-Buddy.git
cd Offline-AI-Buddy
./scripts/fetch-models.sh          # ~5-8 minutes on home Wi-Fi (~1 GB)
brew install xcodegen
./scripts/run-ios-sim.sh
```

This generates the `ios/OfflineAIBuddy.xcodeproj`, builds the Debug app, boots iPhone 15 Simulator, installs it, and launches it. The model is side-loaded into the simulator's documents directory so onboarding skips the download step in dev builds.

60-second Android spin-up

```
cd Offline-AI-Buddy
./scripts/fetch-models.sh
./scripts/run-android-emulator.sh
```

Requires `ANDROID_HOME`, `ANDROID_NDK_HOME`, and at least one created AVD (default `Pixel_6_API_34`; override with `AVD_NAME=...`).

Cloning without the submodule (CI / quick-look)

```
git clone git@github.com:AmericanGroupLLC/Offline-AI-Buddy.git
cd Offline-AI-Buddy
git submodule update --init --recursive # pulls vendor/llama.cpp
```

Until the submodule is pulled, neither the iOS Swift Package nor the Android NDK build will compile.

Dev workflow

```

## Edit shared logic in shared/BuddyAICore/Sources/ ...
## Or shared logic mirror in android/core/src/main/ ...

## Run every test suite locally (skips iOS on non-macOS)
./scripts/test-all.sh

## Just BuddyAICore
cd shared/BuddyAICore && swift test

## Just Android :core
cd android && ./gradlew :core:test

## Just Android :app unit
cd android && ./gradlew :app:testDebugUnitTest

## Tokens/sec benchmark
./scripts/bench-llama.sh

## Marketing site preview
python3 -m http.server # then http://localhost:8000

```

Common gotchas

- **First-launch download:** production builds download ~1 GB on first launch over Wi-Fi. Local dev: pre-fetch with `./scripts/fetch-models.sh` to skip the flow.
- **iOS keyboard extension memory cap:** ~70 MB. The keyboard CANNOT load the LLM directly; it talks to the main app via App Group. See `KEYBOARD.md`.
- **Android NDK:** build requires NDK 26+. Set `ANDROID_NDK_HOME` or the Gradle build will skip the JNI compile and the app will crash at runtime when the LLM service starts.
- **:core is JVM-only:** use `./gradlew :core:test`, NOT `:core:testDebugUnitTest`.
- **Submodule:** `vendor/llama.cpp` is a git submodule. Don't forget `--recursive` on first clone, and `git submodule update --init` if you forgot.
- **Voice STT requires permission:** prompted on first push-to-talk use, not at launch.

RELEASE-VIDEO-SCRIPT

RELEASE-VIDEO-SCRIPT.md

Release Video Script — Offline AI Buddy

This file drives the silent MP4 release video built by the umbrella's `release-video.yml` reusable workflow at tag time.

Each `## Heading | <duration_seconds>` defines one scene. Within the scene, list keys as bullets. Recognised keys:

- `viewport: WIDTHxHEIGHT` (default 1280x720)
- `scroll: <px|%|top|bottom>` (default 0)
- `wait: <seconds|ms>` (dwell time after navigation)
- `caption: <text>` (placeholders `{APP_NAME}` and `{VERSION}`)
- `duration: <seconds>` (override the heading-level duration)

The intro and outro cards (3 s each) are auto-generated from the brand colour in `release.config.json`.

Tagline: On-device AI that never leaves your phone.

Hero — top of marketing site | 4

- viewport: 1280x720
- scroll: top
- wait: 600ms
- caption: {APP_NAME} {VERSION}

Features overview | 5

- viewport: 1280x720
- scroll: 25%
- wait: 600ms
- caption: New in {VERSION}

Highlight reel | 5

- viewport: 1280x720
- scroll: 50%
- wait: 600ms
- caption: Built for everyone

What's next | 5

- viewport: 1280x720
- scroll: 75%
- wait: 600ms
- caption: Available everywhere

Call to action — bottom of page | 4

- viewport: 1280x720
- scroll: bottom
- wait: 800ms
- caption: Try {APP_NAME} today

RELEASING

RELEASING.md

RELEASING.md

Offline AI Buddy's release pipeline is `release.yml`, gated on `pre-release-tests.yml`. Cutting a release is one tag push.

0. Pre-flight

- All keystone tests green locally: `./scripts/test-all.sh`
- Manual smoke checklist passed on a real phone (see `TESTING.md`).
- Marketing site preview reads correctly: `python3 -m http.server`.
- `MODELS.md` SHA-256 entry matches the GGUF you downloaded.

1. Bump version

```
./scripts/bump-version.sh 1.2.0
git diff
```

This updates the iOS `Info.plist` and Android `:app:build.gradle.kts` versions to the new semver.

2. Commit + tag

```
git commit -am 'chore(release): v1.2.0'
git tag v1.2.0
git push origin main v1.2.0
```

Pushing the tag triggers `release.yml`. Pre-release tests must pass before any binary is built or uploaded.

3. What `release.yml` does

Job	Purpose	Gated on
<code>pre-release-tests</code>	Run every test suite (iOS unit + UI, Android unit + UI).	always
<code>build-android</code>	Build APK + AAB. Sign if <code>ANDROID_KEYSTORE_BASE64</code> secret set.	<code>pre-release-tests</code>
<code>build-ios</code>	Build unsigned <code>OfflineAIBuddy.app</code> for iPhone Sim.	<code>pre-release-tests</code>
<code>build-web</code>	Zip the marketing site.	<code>pre-release-tests</code>
<code>publish-github-release</code>	Create the GitHub Release with all artefacts.	all build-*
<code>publish-play-store</code>	Upload AAB to Google Play.	secret <code>PLAY_STORE_SERVICE_ACCOUNT_JSON</code>
<code>publish-testflight</code>	Build signed <code>.ipa</code> + push to TestFlight.	secret <code>APP_STORE_CONNECT_API_KEY_P8_BASE64</code>

4. Required secrets

All optional — workflow no-ops gracefully when unset.

Secret	Purpose
CODECOV_TOKEN	Coverage upload (private repos).
ANDROID_KEYSTORE_BASE64	Sign Play Store AAB.
ANDROID_KEYSTORE_PASSWORD, ANDROID_KEY_ALIAS, ANDROID_KEY_PASSWORD	Sign Play Store AAB.
PLAY_STORE_SERVICE_ACCOUNT_JSON	Upload AAB to Play.
PLAY_STORE_PACKAGE_NAME	com.americangroupllc.offlineaibuddy
APP_STORE_CONNECT_API_KEY_ID, APP_STORE_CONNECT_API_ISSUER_ID, APP_STORE_CONNECT_API_KEY_P8_BASE64	TestFlight upload.
APPLE_TEAM_ID	Code signing.
MATCH_PASSWORD, MATCH_GIT_URL	Fastlane Match (provisioning profile sync).
REVENUECAT_API_KEY_IOS, REVENUECAT_API_KEY_ANDROID	RevenueCat init. Workflow skips RevenueCat-related steps when unset.
ADMOB_APP_ID_IOS, ADMOB_APP_ID_ANDROID	AdMob init. Workflow skips AdMob-related steps when unset.

5. Pre-release tags

Use `-rc.N`, `-beta.N`, `-alpha.N` suffixes:

```
git tag v1.2.0-rc.1
```

These are flagged as pre-releases on GitHub and routed to the `beta` track on Google Play.

6. Hotfix

```
./scripts/bump-version.sh 1.2.1
git commit -am 'fix(chat): Hindi token streaming regression'
git tag v1.2.1
git push origin main v1.2.1
```

7. Local dry-run

Before tagging, validate the build will succeed:

```
./scripts/release-dry-run.sh v1.2.0
```

Stages all artefacts in `distribution/staging-v1.2.0/`. No tag is pushed.

8. Re-pinning the llama.cpp submodule

Bumping `vendor/llama.cpp` is its own commit, reviewed separately from a release:

```
cd vendor/llama.cpp
git fetch && git checkout <new-commit-sha>
cd ../../
## Update MODELS.md $1 with the new pin + verify CI builds.
git commit -am 'chore(deps): bump llama.cpp to <new-commit-sha>'
```

Re-run `./scripts/test-all.sh` to verify both platforms still build.

SAFETY

SAFETY.md

SAFETY.md

How Offline AI Buddy handles content safety, kid-safe mode, and the PIN lock.

§1 — Two profiles

Every install has up to two profiles:

Profile	Default	System prompt	Modes available	Voice preset
Adult	yes	"You are a friendly, helpful, honest assistant..."	All 6 (Chat, Roast, Daily, Party, Game Coach, Translate)	User-selectable
Kid-Safe	optional	strict refusal preamble (see §3)	5 (Roast disabled)	Forced "calm"

Profiles share the same model weights — kid-safe is purely a prompt + filter layer.

§2 — PIN lock

Switching from **Kid** → **Adult** requires a 4-digit PIN, set once during onboarding when the user first selects Kid-safe.

- PIN is stored as **PBKDF2-SHA256** (100,000 rounds, 16-byte random salt) in `profiles.json`. Never stored in plaintext.
- Failed PIN attempts: after 5 failures, the PIN prompt locks for 60 seconds.
- PIN reset: only by deleting and re-creating the Kid-safe profile from the Adult profile (which itself requires the PIN). No "forgot PIN" email — there's no email, there's no server.

Implementation: `BuddyAICore.ProfilesStore.verify(pin:for:)` / `ProfilesStore.kt#verify`.

§3 — Kid-safe system prompt

Prepended to every Kid-safe chat session, in the chat language:

"You are talking with a child. Refuse anything involving violence, weapons, drugs, alcohol, gambling, romance, profanity, self-harm, or dangerous activities. If asked about any of these, say 'Let's pick a different topic — how about [age-appropriate suggestion]?' Always be kind, encouraging, and curious. Keep answers short."

Translated equivalents for hi/zh/fr/es live in `PromptTemplates`.

§4 — Output filter (`ContentPolicy`)

Every assistant token passes through `ContentPolicy.filter(text:)`, which:

1. Maintains a rolling 80-character buffer (regex matches across token boundaries).
2. Applies a curated regex deny-list, cased per-language, covering:
 - Violence keywords (kill, weapon, blood, ...)
 - Romance keywords (kiss, naked, sex, ...)
 - Profanity (full per-language lists)
 - Drugs / gambling / dangerous activities
3. On match, the assistant turn is **truncated** (not edited mid-word — the regex engine returns the offset; we drop everything from the first match onward) and replaced with: "Let's pick a different topic — how about something fun?"

The filter is **idempotent**: filtering already-filtered output is a no-op.

The regex catalogue is in: - `shared/BuddyAICore/Sources/BuddyAICore/Domain/ContentPolicy.swift` - `android/core/src/main/.../core/domain/ContentPolicy.kt`

...both fed by `Tests/ ... /ContentPolicyTests.swift / ContentPolicyKtTest.kt` with 30 cases per language.

§5 — Disabled features in Kid-safe

- **Roast Mode** — entire feature is hidden from the home grid.
- **Daily Challenges** that touch flagged categories — filtered out at template-selection time (`PromptTemplates` knows whether a template is kid-safe).
- **Party Question Generator** — runs against a kid-safe template variant only (no romance/dating themes).
- **Voice TTS** — locked to a “calm” voice (no upbeat / sarcastic presets).

§6 — Audit trail (App Store / Play Store rating questionnaires)

When the App Review or Play Console asks “Does your app contain user-generated open text?” the answer is yes and we link them to this document. Specifically:

- **App Store age rating questionnaire** → “Frequent/Intense Cartoon or Fantasy Violence” = No; “Frequent/Intense Realistic Violence” = No; “Frequent/Intense Sexual Content or Nudity” = No; “Unrestricted Web Access” = No.
- **Google Play content rating** → matches the IARC questionnaire to a Teen rating with the Kid-safe profile noted.

The Kid-safe profile is positioned as a **parental control**, not a substitute for adult supervision. The Consent screen on first launch explicitly says so.

§7 — Why not classifier-based moderation?

A Llama-Guard-style classifier model would be ~500 MB extra on disk and ~2× slower per-token. Regex deny-list + strict system prompt is the right tradeoff for v1; v1.1 may revisit if false-positives are unacceptably high.

§8 — Reporting harmful output

Users can long-press an assistant message → **Report**. The report:

- Stores a local copy of the offending exchange in `Documents/offlineaibuddy/reports/<timestamp>.json`.
- Pre-fills a system mail compose to `safety@americangroupllc.com` (user can edit + send or cancel).
- Never sends without user action.
- Locally displays a “Thanks — we’ll review this in the next app update” toast.

We do not collect telemetry on reports. The local file is the only artifact.

SENTRY

SENTRY.md

SENTRY.md

TL;DR — Sentry is NOT enabled in v1.

Offline AI Buddy is offline-first. There is no Sentry DSN, no init call, no SDK binary in the v1 .ipa or .aab. This document is a forward-looking spec for v1.1 only.

Why we kept the stub

The `canImport`-gated `CrashReportingService` stub from the Card / Drift / BuddyPlay scaffold is preserved so v1.1 can wire Sentry up without an architecture refactor:

```
// shared/BuddyAICore/Sources/BuddyAICore/Observability/CrashReportingService.swift
public protocol CrashReportingService {
    func capture(error: Error, context: [String: String])
    func breadcrumb(_ message: String, category: String)
}

public final class NoopCrashReportingService: CrashReportingService {
    public init() {}
    public func capture(error: Error, context: [String: String]) {}
    public func breadcrumb(_ message: String, category: String) {}
}

#if canImport(Sentry)
import Sentry
public final class SentryCrashReportingService: CrashReportingService { ... }
#endif
```

Same shape on Android (`android/core/src/main/ ... /observability/CrashReportingService.kt`).

v1.1 wiring plan (NOT in scope for v1)

1. Add **opt-in** Settings toggle: “Send anonymous crash reports”. Default OFF.
2. Behind that toggle, init Sentry with `tracesSampleRate = 0`, `enableUserInteractionTracing = false`, `attachScreenshot = false`.
3. Scrub PII: `beforeSend` strips chat content, profile names, PIN hashes from the event payload.
4. Update `PRIVACY.md` to disclose what’s sent and link to Sentry’s privacy doc.
5. Set DSN via env var (CI secret), not hard-coded — same pattern Drift uses.

What stays no-op forever

- **Replays**: never. Our screen content includes user chat content.
- **User identification**: never. Profile IDs are random UUIDs scoped to the device.
- **Performance traces**: only via the Settings opt-in; default OFF.

Why not in v1

- Adds ~2 MB to each platform binary.
- Requires a backend to receive events; we don’t have one.
- Requires a privacy-policy update we don’t want to ship as part of the initial release.

Where the no-SDK choice is enforced

- `release.yml` does NOT set `SENTRY_DSN_*` env vars in v1. (Slot exists for v1.1.)
- `ios/project.yml` does NOT add the Sentry SPM package.
- `android/app/build.gradle.kts` does NOT add the `io.sentry:sentry-android` dependency.
- `Info.plist` and `AndroidManifest.xml` do NOT request crash-related permissions or background modes.

SIGNING

SIGNING.md

Signing — Offline AI Buddy

This document lists every secret used by the release pipeline and the exact code-signing wiring for each platform. Workflows degrade gracefully when secrets are absent (Android → unsigned APK; iOS → unsigned Simulator `.app.zip` ; Desktop → unsigned binaries).

See the canonical secret reference in the umbrella repo: <https://github.com/AmericanGroupLLC/AmericanGroupLLC/blob/main/SECRETS.md>

Android

Secret	Required	Purpose
ANDROID_KEYSTORE_BASE64	optional	Base64-encoded <code>upload.jks</code>
ANDROID_KEYSTORE_PASSWORD	optional	Keystore password
ANDROID_KEY_ALIAS	optional	Key alias
ANDROID_KEY_PASSWORD	optional	Key password
PLAY_STORE_SERVICE_ACCOUNT_JSON	optional	Google Play upload
PLAY_STORE_PACKAGE_NAME	optional	Play Console package

A template is at `keystore.properties.example`.

iOS

The build-ios job takes one of two paths based on whether `APPLE_CERTIFICATE_BASE64` is set:

- **Set** → import `.p12` to a temp keychain, install the provisioning profile, archive `-sdk iphoneos`, generate `exportOptions.plist` from `release.config.json.bundleId` (overridable via `APPLE_BUNDLE_ID`), `xcodebuild -exportArchive`, attach `<AppSlug>-iOS-<version>.ipa` to the GitHub Release.
- **Unset** → unsigned `-sdk iphonesimulator` build attached as `<AppSlug>-iOS-<version>-Simulator.app.zip` (Simulator-only, cannot run on real devices).

Secret	Required	Purpose
APPLE_CERTIFICATE_BASE64	optional	<code>.p12</code> (Distribution)
APPLE_CERTIFICATE_PASSWORD	optional	<code>.p12</code> password
APPLE_KEYCHAIN_PASSWORD	optional	Temp keychain pw
APPLE_PROVISIONING_PROFILE_BASE64	optional	Wildcard mobileprovision (recommended for OfflineAIBuddyKeyboard)
APPLE_TEAM_ID	optional	10-char team ID
APPLE_BUNDLE_ID	optional	Override the default bundle id (<code>com.americangroupllc.offlineaibuddy</code>)
APP_STORE_CONNECT_API_KEY_ID	optional	TestFlight upload
APP_STORE_CONNECT_API_ISSUER_ID	optional	TestFlight upload
APP_STORE_CONNECT_API_KEY_P8_BASE64	optional	TestFlight upload

Provisioning profile shape

The default bundle ID is `com.americangroupllc.offlineabuddy`. Because this app ships with extensions (`OfflineAIBuddyKeyboard`), use a **wildcard** profile such as `com.americangroupllc.offlineabuddy.*` so the profile covers the host app and every extension target. App Store Connect requires explicit IDs for each target, but the *upload step* is gated by a separate secret and only runs after the .ipa is built.

Desktop

The desktop build job runs on Ubuntu and produces:

- Windows NSIS installer `*.exe` (signed only if you add Windows Authenticode in a follow-up — out of scope here).
 - Linux AppImage (unsigned, portable, no install required).
 - macOS .dmg (**unsigned**; Gatekeeper will block on first launch. See `STORE-PACKAGING.md` for the `xattr -cr` bypass and the notarization roadmap).
-

watchOS

The release job builds watchOS as a **standalone Simulator .app.zip** for QA. Submitting a watch app to the App Store requires the watch target to be embedded inside the iOS `.ipa` (watch2-app pattern), which kicks in automatically once `APPLE_CERTIFICATE_BASE64` is set and the Xcode project's iOS target depends on the watch target.

STORE-PACKAGING

STORE-PACKAGING.md

STORE-PACKAGING.md

How Offline AI Buddy v1 ships to the App Store and Google Play.

§1 — App Store (iPhone)

Field	Value
Display name	Offline AI Buddy
Bundle ID	com.americangroupllc.offlineaibuddy
Min OS	iOS 17
Devices	iPhone (no iPad-specific layout in v1)
Category	Productivity
Subcategory	Utilities
Age rating	12+ (open-text AI; kid-safe profile available)
Pricing	Free with IAP (\$4.99/mo subscription, \$19.99 one-time lifetime)

Capabilities used

- **App Group** — `group.com.americangroupllc.offlineaibuddy` declared on the main target AND the keyboard extension; required for the smart-reply IPC.
- **In-App Purchase** — StoreKit via RevenueCat.
- **Microphone + Speech Recognition** — usage descriptions in `Info.plist`; permissions prompted on first push-to-talk use.
- **Background Tasks** — `URLSession` background download for the ~1 GB model.

NOT used

- Push notifications (no APS entitlement).
- Camera, Photos, Contacts, Location, HealthKit, Bluetooth.
- Sign in with Apple (no accounts).

Privacy nutrition label

Section	v1
Data Used to Track You	None
Data Linked to You	None
Data Not Linked to You	None (opt-in v1.1: anonymous usage counts)

App Review notes (CRITICAL — paste into App Review Notes field)

Offline AI Buddy is an on-device LLM app. On first launch the app downloads a ~1 GB language model file over Wi-Fi (size + Wi-Fi requirement disclosed on the Consent screen BEFORE the download starts). After that, all features run offline.

Test account: not required (no accounts). Sandbox IAP: enable a sandbox tester to test the `oab_pro_monthly` subscription and `oab_pro_lifetime` one-time purchase from the Settings → Premium screen. Keyboard extension: enable “Buddy Keyboard” in iOS Settings → General → Keyboard → Keyboards → Add New Keyboard. Then open Messages and tap the globe to switch to Buddy Keyboard. Type any sentence; the candidate strip will populate with 3 AI-generated suggestions (sourced from the main app via App Group). Kid-safe profile: switch profiles from the home screen avatar; PIN defaults to `1234` in TestFlight builds for ease of review.

§2 — Google Play (Android)

Field	Value
Application label	Offline AI Buddy
Package	<code>com.americangroupllc.offlineaibuddy</code>
<code>minSdkVersion</code>	26 (Android 8)
<code>targetSdkVersion</code>	34 (Android 14)
Category	Productivity
Content rating	Teen (open-text AI; kid-safe profile available)
Ads	Yes (interstitial only; no banner ads) — declared in Data Safety.
In-app purchases	Yes — declared in Data Safety.

Permissions declared

See `PRODUCTION.md §3`. Summary: Mic + Internet + Network State + Notifications + ForegroundService + BindInputMethod + Billing only.
No camera, no Bluetooth, no location.

AdMob App ID (release builds)

The `<meta-data android:name="com.google.android.gms.ads.APPLICATION_ID">` entry in `android/app/src/main/AndroidManifest.xml` is wired via the `admobAppId` manifestPlaceholder set in `android/app/build.gradle.kts`. Resolution order:

1. Gradle project property `-PADMOB_APP_ID_ANDROID=ca-app-pub-XXXX ...`
2. Environment variable `ADMOB_APP_ID_ANDROID`
3. Google's official sample/test ID (`ca-app-pub-3940256099942544~3347511713`) — used by CI and local dev builds so the process doesn't crash at startup with the `MobileAdsInitProvider` `IllegalStateException`.

Production releases must pass the real ID. For local release builds:

```
cd android
./gradlew -PADMOB_APP_ID_ANDROID=ca-app-pub-XXXXXXXXXXXXXXXX~YYYYYYYYYY \
:app:bundleRelease
```

In CI, define the `ADMOB_APP_ID_ANDROID` GitHub repository secret. The `.github/workflows/release-binaries-only.yml` workflow forwards it to gradle automatically when present (and warns + falls back to the test ID when it is empty).

Data safety form

Question	Answer
Does your app collect or share any user data?	No (default). v1.1 may add an opt-in usage-count flag.
Does your app encrypt data in transit?	Yes (model download is HTTPS only).
Can users request deletion?	N/A (no remote data; "Erase all chats" + "Delete model" wipe local store).

Play Console release tracks

`release.yml` routes by tag suffix: - `vX.Y.Z` → production - `vX.Y.Z-rc.N` → beta - `vX.Y.Z-beta.N` → beta - `vX.Y.Z-alpha.N` → alpha - `vX.Y.Z-internal.N` → internal

§3 — Marketing assets

Per-store screenshots live in `distribution/screenshots/<platform>/<lang>/` (not committed; generated on demand). Required:

- iOS: 6.7" (iPhone 16 Pro Max), 6.5" (iPhone 11 Pro Max), 5.5" (iPhone 8 Plus).
- Android: phone (1080×1920) — minimum 2, maximum 8.

Localised whatsnew text lives in `distribution/whatsnew/whatsnew-<locale>/whatsnew.txt` for en-US, hi-IN, zh-CN, fr-FR, es-ES.

§4 — What’s new

`distribution/whatsnew/whatsnew-en-US/whatsnew.txt` is uploaded by `release.yml` as the Play Store release notes (English fallback). The other four locales are uploaded the same way to their localised slots. Keep each under 500 chars.

§5 — Bundle size budget

See `PRODUCTION.md §7`. The store binary stays < 50 MB; the ~1 GB model is downloaded on first launch.

§6 — App Review risk flags

Risk	Mitigation
1 GB-on-first-launch download flagged as misleading	Consent screen on first launch states the download size + Wi-Fi requirement BEFORE the download starts. Pasted in App Review Notes (§1).
AI-generated content flagged for moderation	Kid-safe profile + content blocklist + <code>SAFETY.md</code> documents the policy explicitly. App Review Notes link to it.
Keyboard extension flagged for collecting keystrokes	Extension’s <code>RequestsOpenAccess = false</code> ; it cannot reach network. IPC via App Group. Documented in <code>KEYBOARD.md</code> .

Desktop binaries (Electron)

The `build-desktop` job in `.github/workflows/release.yml` uses `electron-builder` on an Ubuntu runner to produce three artifacts attached to every GitHub Release:

Artifact	Signed?	Notes
Offline AI Buddy-Setup-X.Y.Z.exe (NSIS)	no	Windows SmartScreen will prompt on first run; click <i>More info</i> → <i>Run anyway</i> .
Offline AI Buddy-X.Y.Z.AppImage	no	<code>chmod +x</code> then double-click on Linux.
Offline AI Buddy-X.Y.Z.dmg	no	macOS Gatekeeper will refuse. To install:

```
xattr -cr "/Volumes/Offline AI Buddy/Offline AI Buddy.app"  
cp -R "/Volumes/Offline AI Buddy/Offline AI Buddy.app" /Applications/
```

(or *System Settings* → *Privacy & Security* → *Open Anyway*.)

A future change can add a separate `macos-latest` job + Apple Developer cert to produce a properly notarised .dmg without restructuring this workflow.

The Electron shell loads the existing root `index.html` in a `BrowserWindow` with `contextIsolation` enabled. See `desktop/main.js`.

TESTING

TESTING.md

TESTING.md

Shape of the test suite

Layer	Where	Framework	Runs in CI
Apple shared logic	<code>shared/BuddyAICore/Tests/BuddyAICoreTests/</code>	XCTest	<code>ci.yml</code> , <code>ios.yml</code> , <code>pre-release-tests.yml</code>
iOS UI smoke	<code>ios/OfflineAIBuddyUITests/</code>	XCUITest	<code>ios.yml</code> , <code>pre-release-tests.yml</code>
iOS app unit	<code>ios/OfflineAIBuddyTests/</code>	XCTest	<code>ios.yml</code> , <code>pre-release-tests.yml</code>
Android shared logic	<code>android/core/src/test/ ...</code>	JUnit 4 + Truth	<code>ci.yml</code> , <code>pre-release-tests.yml</code>
Android app unit	<code>android/app/src/test/ ...</code>	JUnit 4 + Truth	<code>ci.yml</code> , <code>pre-release-tests.yml</code>
Android UI smoke	<code>android/app/src/androidTest/ ...</code>	Compose UI testing	<code>android.yml</code> , <code>pre-release-tests.yml</code>

The keystone tests (the project's safety net)

Every keystone helper has an XCTest **and** a JUnit twin. They test the same contract on both runtimes so behaviour cannot drift.

Helper	XCTest	JUnit	What it asserts
PromptTemplates	PromptTemplatesTests	PromptTemplatesKtTest	Stable golden output per (kind, language) . ~25 cases. Round-trip stable across runs.
ContentPolicy	ContentPolicyTests	ContentPolicyKtTest	30 cases of safe-vs-blocked text per language; idempotent across rolling buffer; round-trip.
QuotaTracker	QuotaTrackerTests	QuotaTrackerKtTest	10 chats exhausts free; ad-watch grants +5; midnight rollover resets; pro entitlement bypasses.
LanguageDetector	LanguageDetectorTests	LanguageDetectorKtTest	~50 sentences per language correctly detected; mixed-language returns dominant.
TranslateOrchestrator	TranslateOrchestratorTests	TranslateOrchestratorKtTest	Golden 50-sentence-per-pair (5×4=20 pairs) regression suite. Gated behind RUN_TRANSLATION_GOLDEN=1 env var (slow).
ProfilesStore	ProfilesStoreTests	ProfilesStoreKtTest	PIN hash round-trip; PIN-locked profile switch refused without PIN; corrupt JSON falls back to empty.
ModelStore	ModelStoreTests	ModelStoreKtTest	SHA-256 verification; corruption triggers re-download; "delete model" wipes correctly.
LlamaRunner smoke	LlamaRunnerSmokeTests	LlamaJniSmokeKtTest	5-token generation. Gated behind RUN_LLAMA_SMOKE=1 (needs the ~1 GB model on disk).

These eight files MUST compile + test green on both Swift and Kotlin before any UI work for the corresponding feature ships.

Running locally

```
## Everything (auto-skips suites the host can't run)
./scripts/test-all.sh

## Just BuddyAICore
cd shared/BuddyAICore && swift test
## or with coverage on iPhone Sim
xcodebuild test -scheme BuddyAICore-Package \
  -destination 'platform=iOS Simulator,name=iPhone 15,OS=latest' \
  -enableCodeCoverage YES CODE_SIGNING_ALLOWED=NO

## Just Android :core
cd android && ./gradlew :core:test

## Just Android :app unit
cd android && ./gradlew :app:testDebugUnitTest

## Compose UI smoke (requires emulator + adb)
cd android && ./gradlew :app:connectedDebugAndroidTest

## Just XCUITest smoke
cd ios && xcodegen generate
xcodebuild test -project OfflineAIBuddy.xcodeproj -scheme OfflineAIBuddy \
  -sdk iphonesimulator -configuration Debug \
  -destination 'platform=iOS Simulator,name=iPhone 15,OS=latest' \
  CODE_SIGNING_ALLOWED=NO
```

Manual checklist (pre-release)

Scenario	Steps	Pass criterion
First-launch download	Fresh install → Consent → Profile → Model download progresses to 100% on Wi-Fi → SHA-256 matches → chat works in airplane mode.	Download completes, hash verified, chat responds offline.
Multilingual chat	Switch language picker to Hindi → ask “नमस्ते, कैसे हो?” → response in Hindi. Repeat zh / fr / es.	Reply is in target language.
Live translation	Translate screen → English → Hindi → “Hello, how are you?” → Hindi output.	Output is the translation only, no commentary.
Voice push-to-talk	Hold mic → speak → STT renders text → AI streams → tap play → TTS speaks reply.	All four steps complete in the same locale.
iOS keyboard smart-reply	Enable Buddy Keyboard in iOS Settings → open Messages → type “running late” → 3 suggestions in candidate strip → tap one → inserted.	3 chips visible, tap inserts, no crash.
Android keyboard smart-reply	Enable Buddy Keyboard → open WhatsApp → type “running late” → 3 suggestions in IME strip.	Same.
Kid-safe profile	Switch to Kid → ask flagged Q (e.g. about violence) → response refused or filtered. Switch back → PIN prompt.	Refusal text + PIN gate.
Quota + ad unlock	Send 10 chats → 11th blocked → “watch ad for +5” shown → cached ad plays → +5 chats granted.	Counter visible, refill works.
Subscription paywall	Tap Premium → paywall → sandbox subscribe → entitlement updates → ads disappear → quota becomes unlimited.	Entitlement persists across restart.
One-time purchase	Tap “Offline AI Pro Lifetime” → sandbox purchase → entitlement persists across restarts and reinstall (with same Apple/Google ID).	Entitlement restored after reinstall.

Coverage targets

- `BuddyAICore` (Swift): 80%+ on the keystone helpers.
- `:core` (Kotlin): 80%+ on the keystone helpers.
- App layers: best-effort; mostly thin glue.

Coverage is uploaded by CI to Codecov; the report is informational, not a blocker.